Software Requirements Specification (SRS)

Agent Based Vision-Mixer for Video Broadcasts

Version: 1.0

Date: March 2026

Prepared By:

Khushal Das Roll No. 22P-9341

Sami Ullah Roll No. 22P-9323

Supervisor:

Dr. Omar Usman Khan

Department of Computer Science

FAST National University of Computer and Emerging Sciences
(FAST NUCES)

Contents

1 Introduction **3**
- 1.1 Purpose . . . 3
- 1.2 Scope . . . 3
- 1.3 Definitions, Acronyms, and Abbreviations . . . 3
- 1.4 References . . . 4
- 1.5 Overview . . . 5

2 Overall Description **6**
- 2.1 Product Perspective . . . 6
- 2.2 Product Functions . . . 6
- 2.3 User Classes and Characteristics . . . 7
- 2.4 Operating Environment . . . 7
- 2.5 Design and Implementation Constraints . . . 7
- 2.6 Assumptions and Dependencies . . . 8

3 Specific Functional Requirements **9**
- 3.1 Vehicle Detection . . . 9
  - 3.1.1 FR-VD-01: Real-Time Vehicle Detection . . . 9
  - 3.1.2 FR-VD-02: Vehicle Class Filtering . . . 9
  - 3.1.3 FR-VD-03: Confidence Threshold . . . 9
  - 3.1.4 FR-VD-04: Bounding Box Output . . . 9
  - 3.1.5 FR-VD-05: Multi-Camera Parallel Detection . . . 9
- 3.2 Single-Camera Tracking . . . 9
  - 3.2.1 FR-TR-01: Local Track ID Assignment . . . 9
  - 3.2.2 FR-TR-02: Track Persistence Across Frames . . . 9
  - 3.2.3 FR-TR-03: Kalman Filter Prediction . . . 9
  - 3.2.4 FR-TR-04: Stale Track Removal . . . 9
- 3.3 Cross-Camera Re-Identification . . . 10
  - 3.3.1 FR-RI-01: Feature Embedding Extraction . . . 10
  - 3.3.2 FR-RI-02: Cosine Similarity Matching . . . 10
  - 3.3.3 FR-RI-03: Global ID Assignment . . . 10
  - 3.3.4 FR-RI-04: Feature Database Storage . . . 10
  - 3.3.5 FR-RI-05: Camera-Invariant Representation . . . 10
- 3.4 Camera Scoring and Switching . . . 10
  - 3.4.1 FR-CS-01: Camera Score Calculation . . . 10
  - 3.4.2 FR-CS-02: Best Camera Selection . . . 10
  - 3.4.3 FR-CS-03: Hysteresis Threshold . . . 10
  - 3.4.4 FR-CS-04: Frame Buffer Stability . . . 11
  - 3.4.5 FR-CS-05: Overlap Zone Deduplication . . . 11
  - 3.4.6 FR-CS-06: Automatic Fallback . . . 11
- 3.5 Live View User Interface . . . 11
  - 3.5.1 FR-UI-01: Live Feed Display . . . 11
  - 3.5.2 FR-UI-02: Tracking Overlay . . . 11
  - 3.5.3 FR-UI-03: Camera Switch Notification . . . 11
  - 3.5.4 FR-UI-04: Full Video Request . . . 11
- 3.6 Video Stitching . . . 11
  - 3.6.1 FR-VS-01: Clip Collection . . . 11

3.6.2 FR-VS-02: Chronological Ordering . . . 11
3.6.3 FR-VS-03: Overlap Removal . . . 12
3.6.4 FR-VS-04: Video Export . . . 12
3.6.5 FR-VS-05: Graceful Handling of Missing Clips . . . 12
3.7 Email Delivery . . . 12
3.7.1 FR-ED-01: Video Attachment . . . 12
3.7.2 FR-ED-02: Email Transmission . . . 12
3.7.3 FR-ED-03: Delivery Confirmation . . . 12
3.7.4 FR-ED-04: Authentication Error Handling . . . 12

4 Non-Functional Requirements 13
4.1 Performance Requirements . . . 13
4.2 Reliability Requirements . . . 13
4.3 Usability Requirements . . . 13
4.4 Security Requirements . . . 13
4.5 Maintainability Requirements . . . 14
4.6 Portability Requirements . . . 14

5 External Interface Requirements 15
5.1 User Interfaces . . . 15
5.2 Hardware Interfaces . . . 15
5.3 Software Interfaces . . . 15
5.4 Communication Interfaces . . . 16

6 System Constraints and Assumptions 17
6.1 Constraints . . . 17
6.2 Assumptions . . . 17

Appendix A: Use Case Summary 18

Appendix B: Glossary 19

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the **Agent Based Vision-Mixer for Video Broadcasts**, developed as a Final Year Project at FAST National University of Computer and Emerging Sciences (FAST NUCES). The system is designed to track vehicles across a distributed network of surveillance cameras, automatically switch the live view to the best available camera, re-identify vehicles across camera boundaries, and compile and deliver complete journey footage to authorized personnel via email.

1.2 Scope

The Agent Based Vision-Mixer for Video Broadcasts is a software solution that integrates computer vision, deep learning, and intelligent agent-based decision making to perform seamless multi-camera vehicle surveillance. The system:

- Detects vehicles in real time using the YOLOv8 object detection model.
- Tracks vehicles within a single camera view using the DeepSORT tracking algorithm.
- Re-identifies vehicles across different cameras using a ResNet-IBN appearance embedding model.
- Automatically selects and switches to the camera providing the best video quality using a Dynamic Bayesian Network (DBN) based scoring engine.
- Compiles all recorded clips of a vehicle's journey into a single stitched video upon request.
- Delivers the compiled video to a specified recipient via email (SMTP).

The system does not include hardware procurement, camera installation, or network infrastructure setup. It operates on top of existing RTSP-compatible IP camera networks.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification
FYP	Final Year Project
YOLO	You Only Look Once — a real-time object detection algorithm
YOLOv8	Version 8 of the YOLO object detection model by Ultralytics
DeepSORT	Deep Simple Online and Realtime Tracking — a multi-object tracking algorithm
ReID	Re-Identification — matching the same object across different cameras
ResNet-IBN	Residual Network with Instance-Batch Normalization — a backbone model for ReID

Term	Definition
DBN	Dynamic Bayesian Network — used for camera selection scoring
RTSP	Real Time Streaming Protocol — used for IP camera feeds
SMTP	Simple Mail Transfer Protocol — used for email delivery
MTMCT	Multi-Target Multi-Camera Tracking
BBox	Bounding Box — the rectangular region around a detected object
Track ID	A unique identifier assigned to a vehicle within a single camera
Global ID	A unique identifier assigned to a vehicle across the entire camera network
UI	User Interface
API	Application Programming Interface
GPU	Graphics Processing Unit
FPS	Frames Per Second

1.4 References

1. Hou, R., Chang, B., Ma, B., Gu, Q., & Xu, Y. (2023). Dynamic global tracker for online multi-camera multi-vehicle tracking. *Image and Vision Computing*, 136, 104723. https://www.researchgate.net/publication/400061066_Dynamic_global_tracker_for_online_multi_camera_multi_vehicle_tracking

2. Saraswat, R., & Agarwal, R. (2018). Multicamera video stitching surveillance system. *International Journal of Computer Applications*, 182(48), 1–6. https://www.researchgate.net/publication/329704586_Multicamera_Video_Stitching_Surveillance_System

3. Chen, Y., et al. (2025). City-scale multi-camera vehicle tracking system with improved self-supervised camera link model. *ResearchGate Preprint*. https://www.researchgate.net/publication/390288114_City-Scale_Multi-Camera_Vehicle_Tracking_System_with_Improved_Self-Supervised_Camera_Link_Model

4. Al-Nawashi, M., Al-Hazaimeh, O. M., & Saraee, M. (2023). A robust deep networks based multi-object multi-camera tracking system for city scale traffic. *Multimedia Tools and Applications*, 83, 20987–21009. https://link.springer.com/article/10.1007/s11042-023-16243-7

5. Li, J., et al. (2022). Multi-camera vehicle tracking system for AI City Challenge 2022. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pp. 3264–3273. https://openaccess.thecvf.com/content/CVPR2022W/AICity/papers/Li_Multi-Camera_Vehicle_Tracking_System_for_AI_City_Challenge_2022_CVPRW_2022_paper.pdf

6. Zhang, Y., et al. (2025). A real-time and scalable multi-camera vehicle tracking framework powered by edge computing. *arXiv preprint arXiv:2511.13904*. https://arxiv.org/pdf/2511.13904

1.5 Overview

The remainder of this document is organized as follows:

- **Section 2** provides an overall description of the system, including its context, assumptions, and constraints.
- **Section 3** describes the specific functional requirements of the system.
- **Section 4** describes the non-functional requirements including performance, reliability, and security.
- **Section 5** lists external interface requirements.
- **Section 6** describes system constraints and assumptions.

2. Overall Description

2.1 Product Perspective

The Agent Based Vision-Mixer for Video Broadcasts is a standalone software application that interfaces with an existing IP camera network. It operates as an intelligent surveillance layer on top of RTSP video streams. The system is composed of four primary layers:

1. **Detection Layer** — Captures frames from all active cameras and applies YOLOv8 to detect vehicles in real time.
2. **Tracking Layer** — Applies DeepSORT within each camera to assign and maintain local track IDs across frames.
3. **Re-Identification and Switching Layer** — Uses ResNet-IBN embeddings and cosine similarity to match vehicles across cameras, and uses a DBN-based scoring engine to determine the best camera view.
4. **Output Layer** — Stitches recorded clips into a full journey video using FFmpeg and delivers it via email on request.

The system is not a replacement for existing video management systems (VMS) but an intelligent add-on that provides automated tracking and switching capabilities.

2.2 Product Functions

The major functions of the system are summarized below:

ID	Function	Description
F1	Vehicle Detection	Detect vehicles in each camera frame using YOLOv8 in real time.
F2	Single-Camera Tracking	Assign and maintain local track IDs within each camera using DeepSORT.
F3	Feature Extraction	Extract appearance feature embeddings from detected vehicle crops using ResNet-IBN.
F4	Cross-Camera ReID	Match vehicles across different cameras using cosine similarity on feature embeddings.
F5	Camera Scoring	Score all active cameras based on bounding box area, resolution, and coverage quality.
F6	Camera Switching	Automatically switch the active live feed to the highest-scoring camera using DBN.
F7	Overlap Zone Handling	Detect when two cameras cover the same vehicle and select the higher-quality feed.
F8	Live View Display	Display the active camera feed with tracking overlays in the UI.
F9	Clip Storage	Save video clips from each camera to the database with timestamps.
F10	Video Stitching	Compile all clips of a vehicle's journey into a single chronological video using FFmpeg.

ID	Function	Description
F11	Email Delivery	Send the compiled journey video to a specified recipient via SMTP.

2.3 User Classes and Characteristics

User Class	Description
Security Operator	The primary user of the system. Monitors the live view, observes tracking overlays, and requests full journey videos. Assumed to have basic computer literacy but no programming knowledge.
System Administrator	Responsible for configuring camera IP addresses, SMTP settings, resolution weights, and threshold parameters. Has technical knowledge of the system.
Video Recipient	The person who receives the compiled journey video via email. No direct interaction with the system.

2.4 Operating Environment

- **Operating System:** Ubuntu 20.04 LTS or Windows 10/11 (64-bit).
- **Programming Language:** Python 3.9 or later.
- **Hardware:** Minimum Intel Core i5 (8th gen) or equivalent, 16 GB RAM, NVIDIA GPU with CUDA support recommended.
- **Camera Network:** RTSP-compatible IP cameras (minimum 720p resolution).
- **Network:** Local area network (LAN) with stable internet access for email delivery.
- **Database:** SQLite or PostgreSQL for clip metadata and vehicle tracking records.

2.5 Design and Implementation Constraints

- The system must operate at a minimum of 15 FPS per camera stream for real-time tracking.
- All deep learning inference must run within the Python ecosystem using PyTorch and the Ultralytics library.
- The camera switching decision must complete within a single frame cycle (i.e., within $\leq$ 66ms at 15 FPS).
- The system must support a minimum of 3 simultaneous camera streams and scale to at least 10.
- Video stitching must use FFmpeg via Python subprocess or the `ffmpeg-python` library.
- Email delivery must use Python's `smtplib` standard library.

- The system must not require internet access for the core tracking and switching functions.

2.6 Assumptions and Dependencies

- All cameras are pre-installed, calibrated, and accessible via RTSP streams on the local network.
- Camera coverage zones have been mapped in advance (i.e., which cameras overlap and which cameras are adjacent).
- A GPU-enabled machine is available for running YOLOv8 and ResNet-IBN inference in real time.
- The SMTP server credentials provided by the system administrator are valid and accessible.
- The YOLOv8 model weights (`yolov8n.pt` or higher) are pre-downloaded and stored locally.
- The ResNet-IBN ReID model is pre-trained on the VehicleX or VERI-776 dataset.
- The CityFlow or AI City Challenge dataset may be used for system evaluation during testing.

3. Specific Functional Requirements

3.1 Vehicle Detection

3.1.1 FR-VD-01: Real-Time Vehicle Detection

The system shall process each camera frame through the YOLOv8 model and return a list of detected vehicles with bounding boxes, class labels, and confidence scores within a single frame cycle.

3.1.2 FR-VD-02: Vehicle Class Filtering

The system shall filter detection results to include only vehicle classes: car, bus, truck, and motorcycle (COCO class indices 2, 3, 5, 7). Pedestrians, cyclists, and all other objects shall be excluded.

3.1.3 FR-VD-03: Confidence Threshold

The system shall only accept detections with a confidence score greater than or equal to a configurable threshold (default: 0.60).

3.1.4 FR-VD-04: Bounding Box Output

For each detected vehicle, the system shall output a bounding box defined by coordinates (x_1, y_1, x_2, y_2), the class label, and the confidence score.

3.1.5 FR-VD-05: Multi-Camera Parallel Detection

The system shall run vehicle detection in parallel across all active camera streams simultaneously.

3.2 Single-Camera Tracking

3.2.1 FR-TR-01: Local Track ID Assignment

The system shall assign a unique local track ID to each detected vehicle within a single camera using the DeepSORT algorithm.

3.2.2 FR-TR-02: Track Persistence Across Frames

The system shall maintain a vehicle's local track ID across consecutive frames even when the vehicle is briefly occluded, for up to a configurable maximum age (default: 30 frames).

3.2.3 FR-TR-03: Kalman Filter Prediction

The system shall use a Kalman filter to predict the next position of each tracked vehicle when a detection is not available for a given frame.

3.2.4 FR-TR-04: Stale Track Removal

The system shall remove a track from the active tracking list if no matching detection is found for more than the configured maximum age.

3.3 Cross-Camera Re-Identification

3.3.1 FR-RI-01: Feature Embedding Extraction

The system shall extract a fixed-length appearance feature embedding from each detected vehicle crop using the ResNet-IBN model.

3.3.2 FR-RI-02: Cosine Similarity Matching

The system shall compare a new vehicle's embedding against all stored embeddings in the feature database using cosine similarity.

3.3.3 FR-RI-03: Global ID Assignment

If the cosine similarity between a new embedding and an existing one exceeds a configurable threshold (default: 0.75), the system shall assign the matching global vehicle ID to the new track. Otherwise, a new global ID shall be created.

3.3.4 FR-RI-04: Feature Database Storage

The system shall store all feature embeddings and their associated global vehicle IDs in a persistent database for cross-session re-identification.

3.3.5 FR-RI-05: Camera-Invariant Representation

The system shall use the IBN (Instance-Batch Normalization) layers in the backbone to produce appearance embeddings that are robust to changes in camera color tone and brightness across different cameras.

3.4 Camera Scoring and Switching

3.4.1 FR-CS-01: Camera Score Calculation

The system shall calculate a score for each active camera that currently has the tracked vehicle in frame, using the formula:

$$score = \frac{bbox_area}{frame_area} \times w_r$$

where w_r is the resolution weight of the camera, configurable per camera.

3.4.2 FR-CS-02: Best Camera Selection

The system shall select the camera with the highest score as the active feed using a Dynamic Bayesian Network that considers both the current score and the previous active camera's score history.

3.4.3 FR-CS-03: Hysteresis Threshold

The system shall not trigger a camera switch unless the score of the candidate camera exceeds the score of the current active camera by a configurable hysteresis threshold (default: 0.15), to prevent unnecessary feed toggling.

3.4.4 FR-CS-04: Frame Buffer Stability

The system shall require the candidate camera to maintain a higher score than the active camera for a configurable number of consecutive frames (default: 5 frames) before triggering the switch.

3.4.5 FR-CS-05: Overlap Zone Deduplication

When two cameras detect the same vehicle simultaneously in an overlap zone, the system shall select only the higher-scoring camera as the active feed and suppress the duplicate detection from the lower-scoring camera.

3.4.6 FR-CS-06: Automatic Fallback

If the active camera loses the tracked vehicle for more than 3 consecutive frames, the system shall automatically switch to the next best available camera without operator intervention.

3.5 Live View User Interface

3.5.1 FR-UI-01: Live Feed Display

The system shall display the live video stream from the currently active camera in the main view window, updated in real time.

3.5.2 FR-UI-02: Tracking Overlay

The system shall overlay bounding boxes and global vehicle IDs on the live feed for all currently tracked vehicles.

3.5.3 FR-UI-03: Camera Switch Notification

The system shall display a visible notification in the UI when a camera switch occurs, indicating the new active camera ID.

3.5.4 FR-UI-04: Full Video Request

The system shall provide a button or control in the UI that allows the Security Operator to request the full journey video for a selected vehicle.

3.6 Video Stitching

3.6.1 FR-VS-01: Clip Collection

The system shall automatically collect and index all video clips associated with a given global vehicle ID from the clip database.

3.6.2 FR-VS-02: Chronological Ordering

The system shall sort all collected clips by their start timestamp before stitching to ensure the output video is in chronological order.

3.6.3 FR-VS-03: Overlap Removal

The system shall detect and remove duplicate time segments caused by overlap zones before stitching, retaining only the higher-resolution clip for any overlapping time window.

3.6.4 FR-VS-04: Video Export

The system shall use FFmpeg to concatenate all clips into a single output video file in MP4 format with H.264 encoding.

3.6.5 FR-VS-05: Graceful Handling of Missing Clips

If a clip file is missing or corrupted, the system shall log a warning, skip the missing clip, and continue stitching with the remaining valid clips.

3.7 Email Delivery

3.7.1 FR-ED-01: Video Attachment

The system shall attach the stitched journey video file to an outgoing email message.

3.7.2 FR-ED-02: Email Transmission

The system shall send the email with the video attachment to the configured recipient address using the SMTP protocol.

3.7.3 FR-ED-03: Delivery Confirmation

The system shall return a success or failure status after the email transmission attempt and log the result.

3.7.4 FR-ED-04: Authentication Error Handling

If the SMTP authentication fails, the system shall catch the exception, log the error, and notify the operator without crashing the main application.

4. Non-Functional Requirements

4.1 Performance Requirements

ID	Requirement	Description
NFR-P-01	Frame Rate	The system shall process each camera stream at a minimum of 15 FPS during normal operation.
NFR-P-02	Detection Latency	YOLOv8 detection shall complete within 50ms per frame on a GPU-enabled machine.
NFR-P-03	Switching Latency	The camera switching decision shall be made within one frame cycle ($\leq$ 66ms at 15 FPS).
NFR-P-04	ReID Latency	Feature extraction and cosine similarity matching shall complete within 30ms per vehicle crop.
NFR-P-05	Stitching Time	Video stitching for a 10-minute journey shall complete within 60 seconds.
NFR-P-06	Scalability	The system shall support a minimum of 3 and up to 10 simultaneous camera streams.

4.2 Reliability Requirements

- **NFR-R-01:** The system shall maintain continuous tracking for at least 95% of vehicle journeys without losing the global vehicle ID.
- **NFR-R-02:** The system shall recover from a single camera failure without crashing. The tracking pipeline shall continue using the remaining active cameras.
- **NFR-R-03:** The system shall not produce duplicate global vehicle IDs for the same physical vehicle across any camera in the network.
- **NFR-R-04:** The stitched output video shall not contain duplicate time segments from overlapping cameras.

4.3 Usability Requirements

- **NFR-U-01:** The live view UI shall be operable by a Security Operator with no programming knowledge.
- **NFR-U-02:** Camera switches shall be visually smooth and shall not produce abrupt or jarring jumps in the live feed.
- **NFR-U-03:** All status messages, warnings, and error logs shall be displayed in plain English in the system log panel.
- **NFR-U-04:** The full journey video request shall be completable within 3 button clicks from the live view UI.

4.4 Security Requirements

- **NFR-S-01:** SMTP credentials (username, password) shall be stored in an environment variable or encrypted configuration file and shall not be hard-coded in the

source code.

- **NFR-S-02:** Access to the live view UI and video request functions shall be restricted to authenticated users.
- **NFR-S-03:** All video clips and compiled journey videos stored on disk shall be accessible only to authorized system users.
- **NFR-S-04:** The system shall not transmit any vehicle tracking data outside the local network except for the authorized email delivery function.

4.5 Maintainability Requirements

- **NFR-M-01:** The system shall be modular such that any individual component (detector, tracker, ReID module, switching engine, stitcher, email service) can be updated or replaced independently.
- **NFR-M-02:** All configurable parameters (confidence threshold, hysteresis threshold, frame buffer size, SMTP settings) shall be stored in a single external configuration file (e.g., `config.yaml`).
- **NFR-M-03:** The codebase shall follow PEP 8 Python coding standards and include inline documentation for all public functions.

4.6 Portability Requirements

- **NFR-PO-01:** The system shall run on both Ubuntu 20.04 LTS and Windows 10/11 without modification to the core codebase.
- **NFR-PO-02:** All Python dependencies shall be listed in a `requirements.txt` file to allow reproducible environment setup.
- **NFR-PO-03:** The system shall support both CPU-only and GPU-accelerated execution, with reduced performance expected on CPU-only hardware.

5. External Interface Requirements

5.1 User Interfaces

The system shall provide a desktop graphical user interface (GUI) with the following elements:

- A main video panel displaying the live feed from the currently active camera.
- A tracking overlay showing bounding boxes and global vehicle IDs on the live feed.
- A camera status panel showing the list of active cameras and their current scores.
- A log panel displaying real-time system messages, warnings, and errors.
- A vehicle list panel showing all currently tracked global vehicle IDs.
- A "Request Full Video" button that triggers video stitching and email delivery for a selected vehicle.
- A camera switch notification banner displayed when an automatic switch occurs.

5.2 Hardware Interfaces

- **IP Cameras:** The system interfaces with RTSP-compatible IP cameras over the local area network. Camera feeds are accessed using OpenCV's `VideoCapture(rtsp://...)` interface.
- **GPU:** The system optionally uses an NVIDIA GPU via CUDA for accelerating YOLOv8 and ResNet-IBN inference.
- **Storage:** The system reads from and writes to local disk storage for video clip files, the feature database, and the configuration file.

5.3 Software Interfaces

Library / Tool	Version	Purpose
ultralytics	≥ 8.0	YOLOv8 vehicle detection
deep-sort-realtime	≥ 1.3	DeepSORT single-camera tracking
torch	≥ 2.0	PyTorch deep learning framework for ReID model
torchvision	≥ 0.15	Image transforms for ResNet-IBN input preprocessing
opencv-python	≥ 4.8	Frame capture, image processing, and video I/O
ffmpeg-python	≥ 0.2	Video clip stitching and export
smtplib	(stdlib)	Email delivery via SMTP
sqlite3	(stdlib)	Clip metadata and feature database storage
PyYAML	≥ 6.0	Configuration file parsing
numpy	≥ 1.24	Array operations and embedding computations

5.4 Communication Interfaces

- **RTSP (Real Time Streaming Protocol):** Used to receive live video streams from IP cameras over the local network. Port 554 is the standard RTSP port.
- **SMTP (Simple Mail Transfer Protocol):** Used to send compiled journey videos to the designated recipient. Standard ports 465 (SSL) or 587 (TLS) are supported.
- **TCP/IP:** All internal communication between the system modules and the camera network operates over standard TCP/IP.

6. System Constraints and Assumptions

6.1 Constraints

1. The system is constrained to operate within the Python 3.9+ ecosystem. No other programming languages are used in the core pipeline.
2. Real-time performance (15+ FPS per camera) requires GPU-accelerated hardware. CPU-only execution is supported but results in degraded performance.
3. The system does not perform camera installation, calibration, or network configuration. These are prerequisites managed by the System Administrator.
4. Video files stored on disk are limited by available local storage. The System Administrator is responsible for managing storage capacity.
5. Email attachments are limited by the SMTP server's maximum attachment size policy (typically 25MB for standard Gmail SMTP).

6.2 Assumptions

1. All IP cameras in the network are online, calibrated, and streaming via RTSP before the system is started.
2. The overlapping coverage zones between adjacent cameras have been identified and documented before system deployment.
3. The YOLOv8 model weights and the ResNet-IBN ReID model weights are predownloaded and stored at the configured paths.
4. The SMTP email credentials provided are valid and the SMTP server allows programmatic access (e.g., app-specific password for Gmail).
5. Each camera has a unique IP address and a unique camera ID assigned in the configuration file.
6. The vehicle tracking use case is limited to outdoor road surveillance environments. Indoor or highly complex multi-level environments may require additional configuration.

Appendix A: Use Case Summary

UC ID	Use Case Name	Primary Actor	Related FR
UC-01	Monitor live camera feeds	Security Operator	FR-UI-01, FR-UI-02
UC-02	Detect vehicle in frame	Camera Network	FR-VD-01 to FR-VD-05
UC-03	Assign local track ID	Automated System	FR-TR-01, FR-TR-02
UC-04	Extract appearance features	Automated System	FR-RI-01
UC-05	Re-identify vehicle	Automated System	FR-RI-02, FR-RI-03
UC-06	Assign global vehicle ID	Automated System	FR-RI-03, FR-RI-04
UC-07	Score all active cameras	Automated System	FR-CS-01
UC-08	Select best camera view	Automated System	FR-CS-02, FR-CS-03
UC-09	Handle overlap zone	Automated System	FR-CS-05
UC-10	Switch camera feed	Automated System	FR-CS-04, FR-CS-06
UC-11	Update live view UI	Security Operator	FR-UI-01, FR-UI-03
UC-12	Request full journey video	Security Operator	FR-UI-04
UC-13	Stitch all clips	Automated System	FR-VS-01 to FR-VS-05
UC-14	Send video via email	Automated System	FR-ED-01 to FR-ED-04
UC-15	Receive compiled footage	Video Recipient	FR-ED-02

Appendix B: Glossary

Term	Meaning
Appearance Embedding	A high-dimensional numerical vector that represents the visual appearance of a detected vehicle, used for re-identification.
Bounding Box	A rectangular region in a video frame that encloses a detected vehicle.
Camera Switching	The automated process of changing the active live feed from one camera to another when the new camera provides a better view of the tracked vehicle.
Cosine Similarity	A metric measuring the cosine of the angle between two feature vectors, used to compare vehicle appearance embeddings.
Global Vehicle ID	A unique identifier assigned to a physical vehicle that persists across all cameras in the network.
Hysteresis Threshold	A minimum score difference required before a camera switch is triggered, used to prevent unnecessary switching.
Local Track ID	A unique identifier assigned to a vehicle within the view of a single camera.
Overlap Zone	A geographic area that is simultaneously visible to two or more cameras.
Re-Identification	The process of recognizing that a vehicle seen by one camera is the same physical vehicle previously seen by a different camera.
Video Stitching	The process of concatenating multiple short video clips from different cameras into a single continuous video file.

The End

Agent Based Vision-Mixer for Video Broadcasts — FAST NUCES — 2026